

Cvičenie 2: Správca kompaktnej pamäte

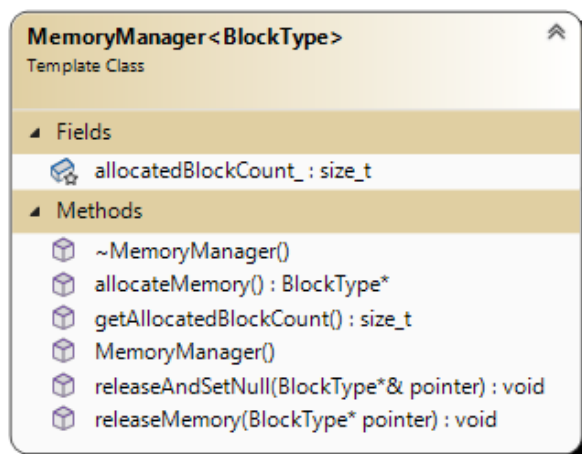
Obsah cvičenia

Správca kompaktnej pamäte prideliť pamäť v lineárnej, súvislej oblasti pamäte. Táto oblasť **musí zostať súvislá** aj po odstránení bloku pamäte z iného miesta ako z jej konca.

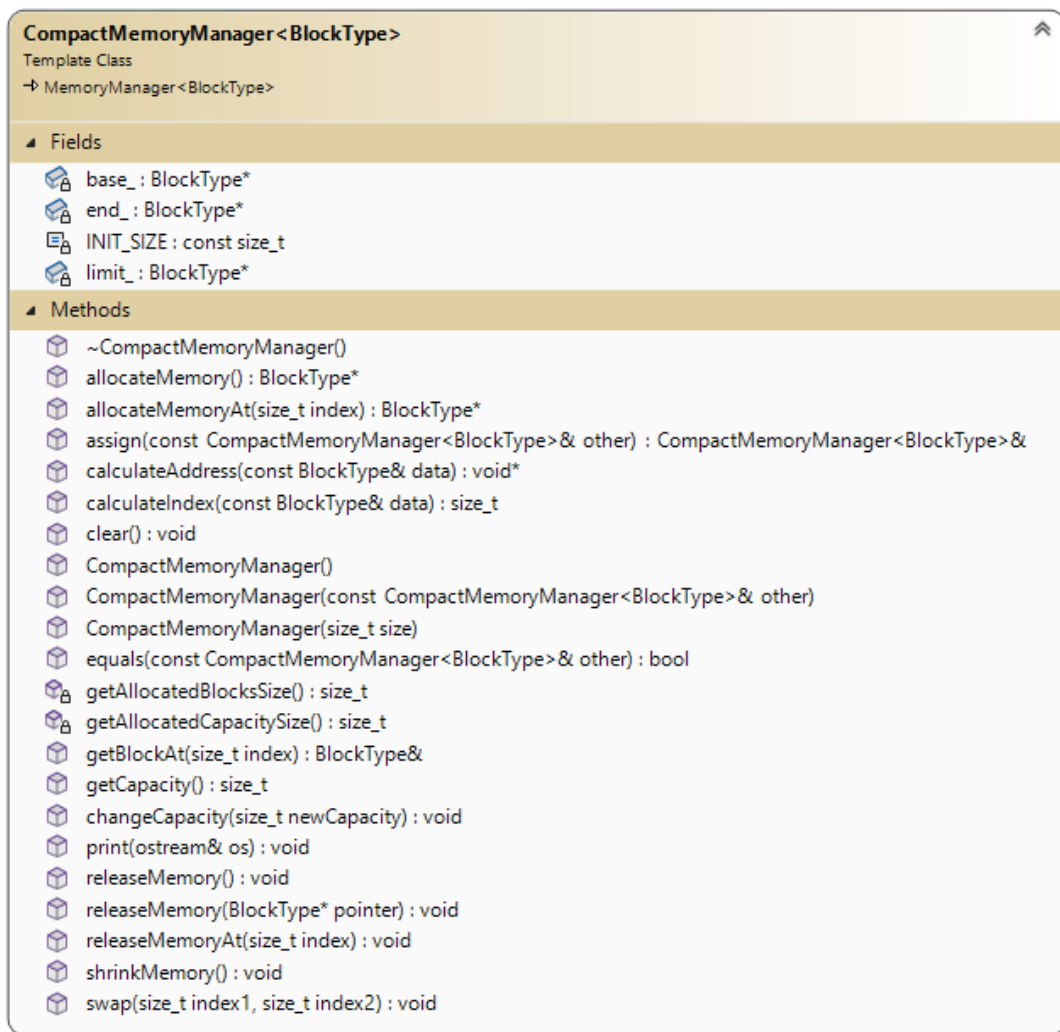
Aj keď z pohľadu správcu pamäte nie je dôležité, čo je v pamäti uložené, z implementačného hľadiska je potrebné brať do úvahy situáciu, kedy sú v pamäti uložené netriviálne objekty. V takomto prípade je potrebné zabezpečiť volanie **konštruktorov** a **deštruktorov** nad alokovanými/dealokovanými pamäťovými miestami.

Správca kompaktnej pamäte implementujeme v triede `CompactMemoryManager` (Obr. 2), ktorá je potomkom triedy `MemoryManager` (Obr. 1). Obe tieto triedy (resp. šablóny tried) preberajú typový parameter `BlockType`, ktorý určuje typ dát, ktoré budú v pamäti uložené. `MemoryManager` zabezpečuje prideliťovanie a uvoľňovanie pamäte v nesúvislej pamäti pomocou operátorov `new` a `delete`. Táto trieda je vo vašich projektoch už implementovaná.

1 UML Diagramy tried

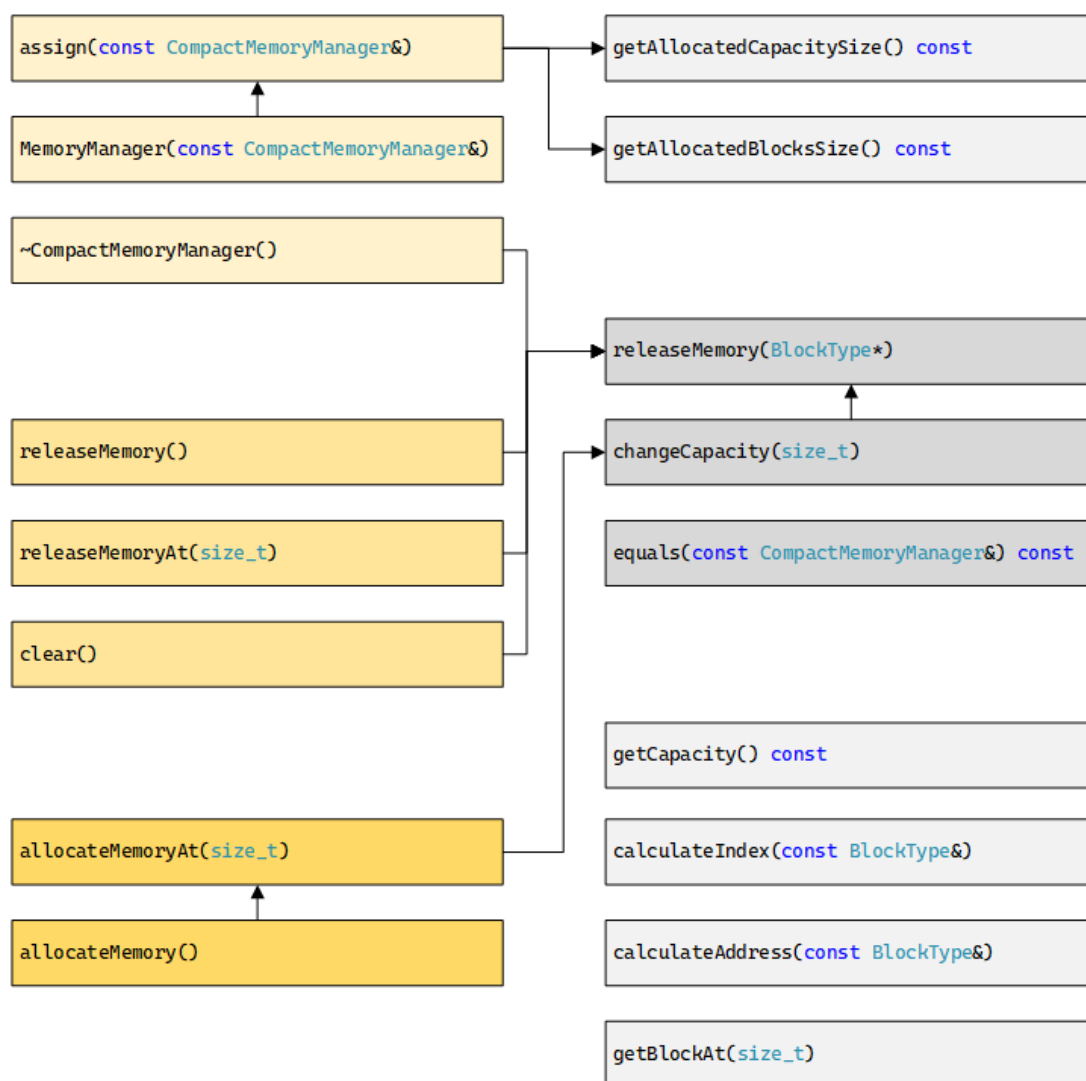


Obr. 1: UML diagram triedy `MemoryManager`

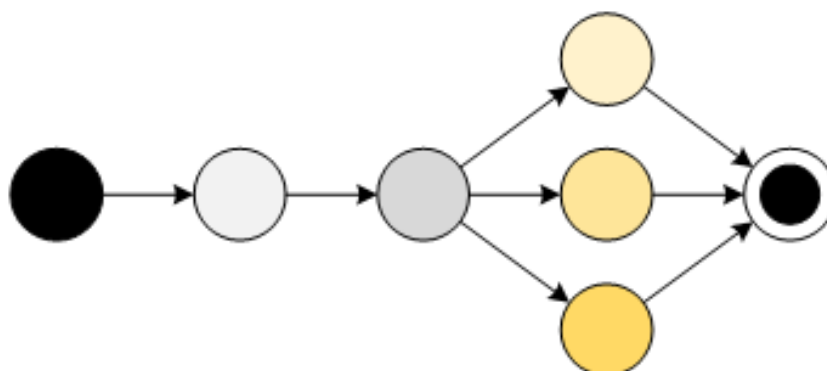
Obr. 2: UML diagram triedy `CompactMemoryManager`

2 Implementácia

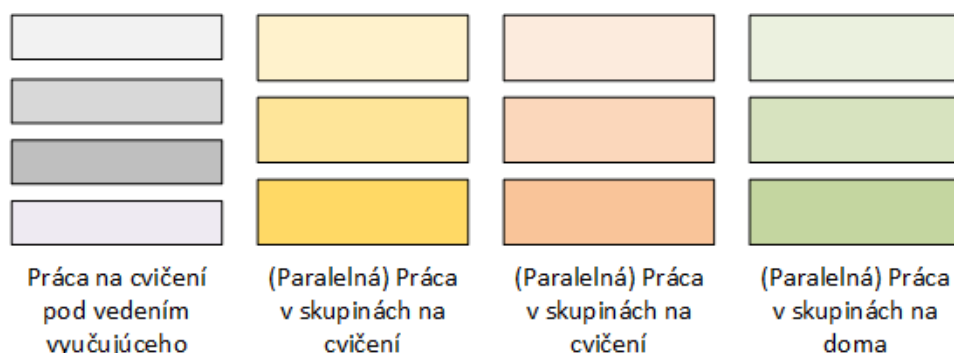
Definície metód správcu kompaktnej pamäte, ktoré je potrebné doplniť nájdete v súbore `libds/mm/compact_memory_manager.h`. Medzi metódami tejto triedy existujú závislosti – niektoré metódy na svoju implementáciu využívajú iné metódy. Vizualizáciu týchto závislostí môžeme vidieť na Obr. 3. Na implementáciu sivých metód v pravej časti nepotrebujeme žiadne iné metódy, preto začneme implementáciou týchto metód – konkrétne metódami v svetlejšej sivej farbe, ktoré implementujú jednoduché **dotazy**. Vizualizáciu poradia, v ktorom budeme metódy implementovať nájdete na Obr. 4, pričom význam farebného označenia nájdete na Obr. 5.



Obr. 3: Grafické znázornenie implementačnej závislosti metód



Obr. 4: Postup implementácie metód



Obr. 5: Význam farieb v grafickom znázornení poradia implementácie

3 Konštruktory a deštruktory

Pri implementácii metód `releaseMemory(BlockType*)` a `releaseMemoryAt(size_t)` bude potrebné explicitne zavolať deštruktor objektu na konkrétnej adrese [↗](#). Okrem syntaxe prezentovanej na uvedenom odkaze môžete pre zjednodušenie využiť pomocnú funkciu `destroy(T*)`.

V metóde `allocateMemoryAt` budeme potrebovať špeciálnu verziu operátora `new` tzv. **placement new**. Pomocou tejto verzie operátora dokážeme dodatočne vytvoriť objekt na už alokovanom pamäťovom mieste poskytnutím ukazovateľa na toto miesto. Dokumentáciu [↗](#) a neformálnu diskusiu [↗](#) k nemu nájdete na priložených odkazoch. Pre zjednodušenie môžete tiež použiť pomocnú funkciu `placement_new(T*)`, ktorá obaľuje volanie operátora `new`. Podobne, pri implementácii metódy `assign` budete potrebovať zavolať operátor `placement new` tak, aby zavolať kopírovací konštruktor vytváraného objektu, na čo môžete vymyslieť pomocnú funkciu `placement_copy(T*, const T&)`.

4 Pseudokódy

Pri implementácii jednotlivých metód si môžete pomôcť pseudokódmi, ktorý nájdete v súbore `compact_memory_manager.h.pdf` medzi súbormi pre dnešné cvičenie na *MS Teams*. Pre zjednodušenie prekladu uvádzame vybrané nízkoúrovňové funkcie na prácu s pamäťou spolu s slovenským názvom použitím v pseudokódoch:

- `std::calloc, std::malloc` – alokujPamäť [↗](#) ;
- `std::realloc` – zmenVeľkosťPamäte [↗](#) ;
- `std::free` – vráťPamäť [↗](#) ;
- `std::memmove` – presuňPamäť [↗](#) ;
- `std::memcmp` – porovnajPamäť [↗](#) .